

Laboratorio di Sistemi Operativi

Programmazione di sistema, Toolchain, System Call UNIX

Processi · File System · File · Segnali · I/O Multiplexing

Luca Argentino
Corso di Laurea in Informatica
Università degli Studi di Bologna
Anno Accademico 2025/2026



Contents

1	Introduzione alla Programmazione di Sistema e alla Toolchain	2
1.1	Cos'è la programmazione di sistema	2
1.2	La toolchain di compilazione	2
1.2.1	Fase 1: Preprocessore (<code>gcc -E</code>)	3
1.2.2	Fase 2: Compilatore (<code>gcc -S</code>)	3
1.2.3	Fase 3: Assemblatore (<code>gcc -c</code>)	3
1.2.4	Fase 4: Linker	3
1.2.5	L'ottimizzatore	3
1.3	Strumenti di ispezione	4
1.4	Il vero punto d'ingresso: <code>_start</code> e il runtime C	4
1.4.1	Implementare <code>_start</code> manualmente in assembly	4
1.5	System call dirette senza <code>libc</code>	5
1.6	Programmazione bare-metal vs Linux	6
2	Le System Call in UNIX: Gestione dei Processi	7
2.1	Cosa sono le system call	7
2.2	Identificazione dei processi: <code>getpid</code> e <code>getppid</code>	7
2.3	<code>fork()</code> : creare un processo figlio	7
2.4	<code>_exit()</code> : terminazione immediata	8
2.5	<code>sleep()</code> : esecuzione asincrona padre-figlio	9
2.6	<code>wait()</code> : sincronizzazione padre-figlio	9
2.7	Processi zombie e orfani	10
2.7.1	Processi zombie	10
2.7.2	Processi orfani	11
2.8	<code>execve()</code> : eseguire un nuovo programma	11
2.8.1	Passare variabili d'ambiente personalizzate	12
2.8.2	La mini-shell	12
2.8.3	La famiglia <code>exec*</code>	13
2.9	Gestione degli errori: <code>errno</code> e <code>perror</code>	13
3	Le System Call in UNIX: Gestione del File System	13
3.1	Introduzione	14
3.2	La system call <code>stat()</code>	14
3.3	<code>stat()</code> vs <code>lstat()</code> : la differenza con i symlink	15
3.4	Lettura delle directory con <code>dirent</code>	15
3.4.1	<code>readdir()</code> : versione minimale di <code>ls</code>	15
3.4.2	<code>scandir()</code> : lettura ordinata	16
4	Le System Call in UNIX: Gestione dei File	16
4.1	Il concetto di File Descriptor	17
4.2	Le syscall fondamentali: <code>open</code> , <code>read</code> , <code>write</code> , <code>close</code>	17
4.2.1	Esempio completo: copia di un file	17
4.3	<code>lseek()</code> : spostarsi nel file e i file sparsi	18
4.4	<code>dup</code> , <code>dup2</code> , <code>dup3</code> : duplicazione e ridirezione	18
4.5	File aperti dopo <code>fork()</code> e <code>exec()</code>	19
4.6	Le pipe	19

4.6.1	Pipe semplice	20
4.6.2	Pipe con fork e ridirezione: implementare <code>ls sort -r</code>	20
4.7	<code>ioctl</code> : controllo dei dispositivi	21
5	Introduzione ai Segnali	21
5.1	Cosa sono i segnali	21
5.2	<code>kill</code> : inviare segnali	21
5.3	<code>signal</code> : registrare un handler	22
5.4	Segnali da eccezioni hardware	23
5.4.1	SIGSEGV — Segmentation Fault	23
5.4.2	SIGFPE — Errore aritmetico	23
5.4.3	SIGINT — Intercettare Ctrl+C	24
5.5	<code>sigaction</code> : gestione avanzata dei segnali	24
6	Attesa di eventi: I/O Multiplexing	25
6.1	Il problema: blocco su read	25
6.2	Evoluzione storica	25
6.3	La system call <code>poll()</code>	25
6.4	Esempio completo: bridge tra file e standard I/O	26

1 Introduzione alla Programmazione di Sistema e alla Toolchain

1.1 Cos'è la programmazione di sistema

La **programmazione di sistema** è lo sviluppo di software che interagisce direttamente con il sistema operativo o con l'hardware. La differenza rispetto alla normale programmazione applicativa è fondamentale:

- La **programmazione applicativa** usa librerie ad alto livello (es. `printf`, `fopen`) che nascondono i dettagli del sistema operativo.
- La **programmazione di sistema** lavora vicino al kernel tramite le **system call**: le vere interfacce che permettono ai processi di richiedere servizi al sistema operativo (gestione file, processi, memoria, rete).

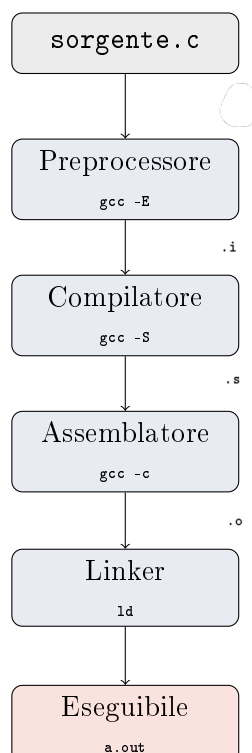
Per padroneggiare questo livello è essenziale capire come il codice sorgente diventa un eseguibile binario, cioè conoscere la **toolchain di compilazione**.

1.2 La toolchain di compilazione

Quando si scrive:

```
$ gcc -o hello_world hello_world.c
```

non si sta semplicemente “compilando”. `gcc` agisce come un **driver**: un orchestratore che invoca in sequenza una serie di strumenti (tool), trasformando il file sorgente in un eseguibile binario attraverso 4 fasi distinte.



1.2.1 Fase 1: Preprocessore (gcc -E)

Il preprocessore elabora le **direttive** del C prima ancora di iniziare la compilazione:

- Espande le macro definite con **#define**
- Sostituisce i file inclusi con **#include** (letteralmente incolla il contenuto)
- Gestisce le direttive condizionali (**#ifdef**, **#ifndef**, **#endif**)
- Rimuove tutti i commenti

Il risultato è un file **.i**: un file C “espanso” senza direttive, pronto per il compilatore.

```
$ gcc -E hello_world.c      # produce hello_world.i (spesso molto  
lungo)
```

1.2.2 Fase 2: Compilatore (gcc -S)

Il compilatore vero e proprio analizza il codice preprocessato, verifica la correttezza sintattica e semantica, e lo traduce in **linguaggio assembly** (mnemonici human-readable per le istruzioni macchina).

```
$ gcc -S hello_world.c      # produce hello_world.s (testo assembly)
```

1.2.3 Fase 3: Assemblatore (gcc -c)

L’assemblatore converte il file assembly in **codice macchina binario**, producendo un **file oggetto .o**. Questo file contiene istruzioni macchina già eseguibili, ma non è ancora un programma completo: i riferimenti a simboli esterni (come funzioni di libreria) non sono ancora risolti — sono dei “buchi” da riempire.

```
$ gcc -c hello_world.c      # produce hello_world.o (binario, non  
eseguibile)
```

1.2.4 Fase 4: Linker

Il linker unisce uno o più file **.o** con le librerie necessarie (es. la **libc**), **risolve tutti i simboli mancanti** e produce l’eseguibile finale. È qui che funzioni come **printf** o **malloc** trovano il loro corpo effettivo.

```
$ gcc hello_world.o  
$ ./a.out
```

1.2.5 L’ottimizzatore

GCC può applicare ottimizzazioni al codice prima o durante la compilazione:

```
$ gcc -O2 -o hello_world hello_world.c      # ottimizzazione media/  
alta
```

Concetto chiave

Con ottimizzazioni elevate, il compilatore può **eliminare codice** che ritiene non produca effetti osservabili. Per esempio, un loop che incrementa una variabile senza usarla all'esterno può essere completamente rimosso dal binario finale. Per impedire questa eliminazione, si dichiara la variabile **volatile**:

```
1 volatile int i;
2 for (i = 0; i < 1000000000; i++); // il loop NON verra'
    eliminato
```

La parola chiave **volatile** dice al compilatore: *“il valore di questa variabile può cambiare per ragioni esterne che non sei in grado di vedere (es. hardware), quindi non ottimizzarla.”*

1.3 Strumenti di ispezione

Dopo aver prodotto file oggetto ed eseguibili, è utile poterli ispezionare:

```
$ file hello_world      # tipo del file (testo, oggetto, eseguibile
                        ELF...)
$ nm hello_world.o      # lista simboli definiti e non nel file
                        oggetto
$ nm a.out              # come sopra per l'eseguibile
```

nm mostra ogni simbolo con una lettera che indica il tipo: T = definito nel segmento testo, U = indefinito (da risolvere dal linker), D = dati inizializzati, ecc.

1.4 Il vero punto d'ingresso: `_start` e il runtime C

Concetto chiave

Il kernel Linux non chiama mai `main()`.

Quando il kernel carica ed esegue un file ELF, salta all'indirizzo indicato come **entry point** nel file. Tale entry point **non** è `main()`, bensì `_start`, una funzione fornita dalla **CRT** (C Runtime, file come `crt1.o`, `crti.o`, `crtn.o`).

La sequenza reale di esecuzione è:

kernel → `_start` → `main()` → `exit()` → kernel

Quando si compila con `gcc hello.c -o hello`, GCC include automaticamente la CRT. Se si prova a linkare manualmente senza di essa, il linker si lamenta:

```
$ gcc -c 42.c          # produce 42.o
$ ld 42.o              # errore!
# ld: warning: cannot find entry symbol _start; defaulting to 0x0
```

1.4.1 Implementare `_start` manualmente in assembly

È possibile definire `_start` a mano, eliminando completamente la dipendenza dalla lib. La funzione deve svolgere tre operazioni secondo l'ABI System V x86-64:

1. Leggere `argc`, `argv` e `envp` dallo stack
2. Chiamare `main(argc, argv, envp)`
3. Invocare la syscall `exit` (numero 60) con il valore di ritorno

```

1 .text
2 .globl _start
3 _start:
4     xor    %ebp, %ebp           # azzera il frame pointer (convenzione
                                # ABI)
5     mov    (%rsp), %edi         # argc = primo valore sullo stack
6     lea    8(%rsp), %rsi        # argv = indirizzo del secondo
                                # elemento
7     lea    16(%rsp,%rdi,8), %rdx # envp = indirizzo dopo argv[argc]
8     call   main                 # chiama main(argc, argv, envp)
9     mov    %eax, %edi           # valore di ritorno di main ->
                                # argomento exit
10    mov    $60, %eax            # syscall number 60 = exit
11    syscall                     # invoca il kernel

```

Listing 1: start.s — implementazione manuale di `_start`

Compilazione e link **senza** libc:

```

$ gcc -c 42.c           # compila il programma C -> 42.o
$ as -o start.o start.s # assembla _start -> start.o
$ ld -o a.out start.o 42.o --entry=_start
$ ./a.out && echo $?    # output: 42

```

1.5 System call dirette senza libc

È possibile scrivere un programma C completo che parla **direttamente** con il kernel senza usare alcuna funzione della libc, invocando le syscall tramite l'istruzione `syscall`.

Secondo l'ABI Linux x86-64, per invocare una syscall si caricano i parametri nei registri:

Registro	Significato
<code>rax</code>	Numero della syscall
<code>rdi</code>	Primo argomento
<code>rsi</code>	Secondo argomento
<code>rdx</code>	Terzo argomento
<code>r10</code>	Quarto argomento
<code>r8</code>	Quinto argomento
<code>r9</code>	Sesto argomento

```

1 long mystrlen(char *s) {
2     long len;
3     for (len = 0; *s != 0; len++) s++;
4     return len;

```

```

5 }
6
7 long mywrite(char *s) {
8     long addr = (long)s;
9     long len  = mystrlen(s);
10    register long r_syscallno asm("rax") = 1; // syscall 1 = write
11    register long r_arg1      asm("rdi") = 1; // fd = 1 (stdout)
12    register long r_arg2      asm("rsi") = addr;
13    register long r_arg3      asm("rdx") = len;
14    register long r_retvalue   asm("rax");
15    asm("syscall"); // INVoca KERNEL
16    return r_retvalue;
17 }
18
19 void myexit(long value) {
20     register long r_syscallno asm("rax") = 60; // syscall 60 = exit
21     register long r_arg1      asm("rdi") = value;
22     asm("syscall");
23 }
24
25 void _start(void) {
26     long retvalue = mywrite("hello world\n");
27     myexit(retvalue); // exit code = numero di byte scritti (12)
28 }

```

Listing 2: hw6.c — programma senza libc

```

$ gcc -c hw6.c      # produce hw6.o (con _start definita)
$ ld hw6.o          # nessun errore: _start e' gia' nel file
$ ./a.out
hello world
$ echo $?
12                  # 12 = numero di caratteri scritti dalla syscall
write

```

1.6 Programmazione bare-metal vs Linux

Concetto chiave

Su Linux, ogni processo vive in uno **spazio di indirizzi virtuale**. Il kernel impedisce l'accesso diretto alla memoria fisica o ai registri hardware. Tentare di accedere a indirizzi fisici come 0x27 o 0x28 (registri dell'ATmega168) causa un **segmentation fault**.

Su un **microcontrollore AVR** invece non esiste kernel, non esiste memoria virtuale, e il codice gira direttamente in modalità privilegiata: quegli indirizzi sono i veri registri hardware. Questo è il concetto di **bare-metal programming**: il codice gira “a metallo nudo”, senza alcun livello di astrazione.

```

# Compilazione per AVR (NON per x86!)
$ avr-gcc -mmcu=atmega168 -Os -o noSO noSO.c

```



```
$ avr-objcopy -O ihex noS0 noS0.hex
$ avrdude -p m168 -c stk500v2 -P /dev/ttyUSB0 -U flash:w:noS0.hex
```

2 Le System Call in UNIX: Gestione dei Processi

2.1 Cosa sono le system call

Le **system call** sono l'interfaccia attraverso cui un processo in modalità utente richiede servizi al kernel. Il processo non può accedere direttamente alle risorse hardware; deve chiedere al kernel di farlo per conto suo.

Nel linguaggio C, le system call sono accessibili tramite funzioni wrapper delle librerie standard, principalmente:

- `<unistd.h>` — processi, file, I/O
- `<sys/wait.h>` — attesa processi figli
- `<sys/stat.h>` — informazioni file

2.2 Identificazione dei processi: getpid e getppid

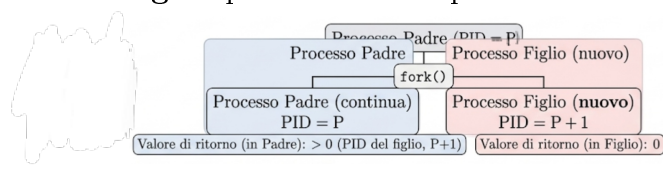
Ogni processo UNIX è identificato da due numeri:

- **PID** (*Process ID*): identificativo univoco nel sistema in un dato momento
- **PPID** (*Parent Process ID*): il PID del processo padre

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("PID: %d, PPID: %d\n", getpid(), getppid());
6     return 0;
7 }
```

2.3 fork(): creare un processo figlio

`fork()` è la syscall fondamentale per la creazione di processi. Duplica il processo corrente, creando un **processo figlio** quasi identico al padre.



Valori di ritorno di `fork()`:

- `> 0` nel padre: contiene il PID del figlio appena creato
- `0` nel figlio

- -1 in caso di errore (troppi processi nel sistema)

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("pre %d (%d)\n", getpid(), getppid());
6
7     if (fork()) {
8         // RAMO PADRE: fork restituisce il PID del figlio (> 0)
9         printf("parent %d\n", getpid());
10    } else {
11        // RAMO FIGLIO: fork restituisce 0
12        printf("child %d (%d)\n", getpid(), getppid());
13    }
14    return 0;
15 }
```

Nel processo Padre: La `fork()` restituisce un numero maggiore di zero (il vero e proprio PID del figlio, per esempio 77978). Il codice per il padre diventa idealmente `if (77978)`. Poiché questo numero non è zero, il C lo valuta come **Vero**, e il padre esegue il blocco di codice dentro l'`if`. 

Esempio pratico

Output tipico (l'ordine non e' garantito):

```

pre 77977 (75992)      # eseguito UNA VOLTA prima della fork
parent 77977          # eseguito dal padre
child 77978 (77977)   # eseguito dal figlio (PPID = PID del padre
)
```

La riga `pre` appare solo una volta perché viene eseguita prima della `fork()`, quando esiste ancora un solo processo.

Nota

L'ordine di esecuzione tra padre e figlio è **non deterministico**. Lo scheduler del kernel decide autonomamente quale dei due eseguire per primo. Non fare mai assunzioni sull'ordine.

2.4 `_exit()`: terminazione immediata

`_exit()` termina il processo **immediatamente**, restituendo al kernel il codice di uscita passato come argomento.

Si distingue da `exit()` perché:

- **Non** esegue il flush dei buffer di I/O della stdio
- **Non** chiude automaticamente i file gestiti dalla stdio
- **Non** invoca le funzioni registrate con `atexit()`

Nei processi figlio si preferisce `_exit()` per evitare il doppio flush dei buffer ereditati dal padre (che potrebbe causare output duplicato).

```

1 _exit(42);           // il codice di uscita sara' 42
2 // echo $? --> 42
```

2.5 sleep(): esecuzione asincrona padre-figlio

`sleep(n)` sospende il processo per `n` secondi. Quando usata dopo una `fork()`, permette di osservare chiaramente la natura asincrona dei due processi:

```
1 if (fork()) {
2     printf("parent %d\n", getpid());
3     // il padre termina subito
4 } else {
5     sleep(2);           // il figlio aspetta 2 secondi
6     printf("child %d (%d)\n", getpid(), getppid());
7 }
```

Esempio pratico

```
$ ./a.out
pre 91231 (90210)
parent 91231
$          <-- la shell torna subito (padre terminato)
child 91232 (1)  <-- appare dopo 2 secondi
                  # PPID = 1 perche' il padre e' gia' morto!
```

Il PPID del figlio è diventato 1 perché il padre è terminato prima: il figlio è diventato **orfano** e è stato adottato da `init` (PID 1).

2.6 wait(): sincronizzazione padre-figlio

`wait()` sospende il processo padre finché **uno dei suoi figli** non termina. Serve a raccogliere lo stato di uscita del figlio ed è fondamentale per evitare i processi zombie.

```
1 #include <sys/wait.h>
2
3 int status;
4 pid_t stchild;
5
6 if (fork()) {
7     // PADRE
8     printf("parent %d\n", getpid());
9     stchild = wait(&status); // si blocca finche' il figlio non
10                             muore
11     printf("%d terminated (%d)\n", stchild, WEXITSTATUS(status));
12 } else {
13     // FIGLIO
14     sleep(2);
15     printf("child %d (%d)\n", getpid(), getppid());
16     _exit(0);
17 }
```

Esempio pratico

```
$ ./a.out
pre 79133 (78120)
parent 79133
child 79134 (79133)      # dopo 2 secondi
79134 terminated (0)     # il padre raccoglie l'exit code 0
```

La macro `WEXITSTATUS(status)` estrae il codice di uscita dagli 8 bit meno significativi del valore `status`. Se il figlio avesse fatto `_exit(5)`, il padre avrebbe ricevuto `terminated (5)`.

2.7 Processi zombie e orfani

2.7.1 Processi zombie

Concetto chiave

Un **processo zombie** è un figlio che ha già terminato l'esecuzione, ma il cui stato di uscita non è stato ancora raccolto dal padre tramite `wait()`. Il kernel conserva alcune informazioni del processo (exit code, segnali ricevuti) finché il padre non le preleva.

```
1 if (fork()) {
2     sleep(60);           // il padre dorme 60 secondi SENZA
                           // chiamare wait
3     wait(&status);       // raccoglie lo zombie dopo 60s
4 } else {
5     printf("child %d\n", getpid());
6     _exit(42);           // il figlio termina subito -> diventa
                           // zombie
7 }
```

Durante la `sleep(60)`, il figlio è uno zombie. Si può verificare con:

```
$ ps ax -o pid,ppid,state,cmd | grep a.out
79134 79133  Z+  [a.out] <defunct>      # Z = zombie, <defunct>
```

Caratteristiche dei zombie:

- Consumano pochissima memoria ma occupano una voce nella process table
- Scompaiono non appena il padre chiama `wait()`
- Se il padre muore senza aver chiamato `wait()`, il figlio zombie viene adottato da `init` che esegue `wait()` al suo posto, eliminandolo

Esempio di output possibile:

```
pre 81231 (80120)
child 81232 (81231)
parent 81231
81232 terminated (42)
post 81231
post 81232
```

2.7.2 Processi orfani

Concetto chiave

Un processo diventa **orfano** quando il padre termina prima di lui. Il kernel reagisce immediatamente adottandolo sotto `init` (PID 1), che chiama automaticamente `wait()` quando il processo terminerà.

```
1 if (fork()) {
2     _exit(0);    // il padre termina subito
3 } else {
4     sleep(5);
5     printf("Nuovo padre: %d\n", getppid()); // stampa: 1
6 }
```

Esempio di output

```
pre 81231 (80120)
parent 81231
child 81232 (81231)
Nuovo padre del figlio 81232: 1
```

Zombie	Orfano
Il figlio termina prima del padre	Il padre termina prima del figlio
Il padre è vivo ma non chiama <code>wait()</code>	Il figlio continua a vivere
Il figlio rimane nella process table	Il figlio viene adottato da <code>init</code>
PPID del figlio non cambia	PPID del figlio diventa 1

2.8 `execve()`: eseguire un nuovo programma

`execve()` **sostituisce completamente** l'immagine del processo corrente con un nuovo programma. Il PID rimane invariato, ma codice, dati, stack e heap vengono rimpiazzati.

```
1 int execve(const char *pathname,
2           char *const argv[],
3           char *const envp[]);
```

- `pathname`: percorso assoluto del programma da eseguire
- `argv[]`: array di stringhe degli argomenti; `argv[0]` è il nome del programma; l'array deve terminare con `NULL`
- `envp[]`: variabili d'ambiente nel formato "CHIAVE=valore", terminato da `NULL`

Nota

Se `execve()` ha successo, **non ritorna mai**. Il codice dopo la chiamata non viene mai eseguito. Se ritorna, significa che c'è stato un errore.

```
1 int main() {
2     char *argv[] = {"/bin/ls", "-l", "/", NULL};
3     execve(argv[0], argv, NULL);
4     // questa riga NON viene mai raggiunta se execve ha successo
5     printf("execve fallita!\n");
6     return 1;
}
```

Esecuzione da bash: (stessa cosa di fare `ls -l`)

```
$ gcc -o prova_ls prova_ls.c
$ ./prova_ls
bin boot dev etc home lib ...
```

Listing 3: Eseguire `/bin/ls` con `execve`

2.8.1 Passare variabili d'ambiente personalizzate

```

1 int main() {
2     char *exec_argv[] = {"bash", NULL};
3     char *env_argv[]  = {"USER=renzox", "prova=sistemi operativi",
4                           NULL};

5     printf("prima execve, pid = %d\n", getpid());
6     execve("/bin/bash", exec_argv, env_argv);
7     // se siamo qui, execve e' fallita
8 }

```

Esecuzione da bash:

```

$ gcc -o prova_bash prova_bash.c
$ ./prova_bash
prima execve, pid = 12345
$ echo $USER
renzox
$ echo $prova
sistemi operativi

```

Dopo l'esecuzione, la shell bash avviata avrà **solo** le variabili `USER` e `prova` nel suo ambiente. Il PID non cambia.

2.8.2 La mini-shell

La combinazione `fork()` + `execve()` + `wait()` è alla base di tutte le shell UNIX:

```

1 int main() {
2     char cmd[1024];
3     for (;;) {
4         if (scanf("%s", cmd) == EOF) break;

5
6         if (fork()) {
7             // PADRE: aspetta la fine del figlio
8             int status;
9             pid_t child = wait(&status);
10            printf("%d terminated (%d)\n", child, WEXITSTATUS(
11                status));
12        } else {
13            // FIGLIO: esegue il comando
14            char *exec_argv[] = {cmd, NULL};
15            execve(cmd, exec_argv, NULL);
16            _exit(-1); // se execve fallisce
17        }
18    }
19 }

```

Listing 4: Mini-shell: `fork` + `exec` + `wait`

Esempio pratico

```

$ ./mini_shell
/bin/ls          # input utente
file1 file2 file3
12347 terminated (0)

```

```
/bin/echo          # input utente
hello world
12348 terminated (0)
CTRL+D            # EOF: chiude la mini-shell
```

2.8.3 La famiglia `exec*`

Tutte le varianti della famiglia `exec` chiamano internamente `execve()`. Differiscono per come si passano gli argomenti e l'ambiente:

Funzione	Caratteristica
<code>execve(path, argv, envp)</code>	Syscall base; tutto esplicito
<code>execv(path, argv)</code>	Eredita l'ambiente corrente
<code>execvp(file, argv)</code>	Cerca in PATH; eredita ambiente
<code>execvpe(file, argv, envp)</code>	Cerca in PATH + envp esplicito
<code>execl(path, arg0, ..., NULL)</code>	Argomenti come lista variadica
<code>execlp(file, arg0, ..., NULL)</code>	Lista variadica + ricerca in PATH
<code>execle(path, arg0, ..., NULL, envp)</code>	Lista variadica + envp esplicito

2.9 Gestione degli errori: `errno` e `perror`

`errno` è una variabile globale (in `<errno.h>`) che contiene il codice dell'ultimo errore. Viene impostata solo in caso di fallimento; non viene azzerata al successo.

```
1 #include <errno.h>
2
3 int err = execvp(argv[1], argv + 1);
4 // execvp ritorna SOLO in caso di errore
5 printf("err = %d, errno = %d\n", err, errno);
6 if (err == -1)
7     perror("execerr"); // stampa: "execerr: No such file or
                        // directory"
```

Esempio pratico

```
$ ./prova_err /bin/lxx
err = -1 errno = 2
execerr: No such file or directory
# errno 2 = ENOENT (file non trovato)
```

3 Le System Call in UNIX: Gestione del File System

3.1 Introduzione

In UNIX tutto è un file. Ogni file, directory o dispositivo è rappresentato da un **inode** — una struttura dati nel kernel che contiene tutti i metadati del file (permessi, proprietario, dimensione, timestamp, posizione dei blocchi su disco). Il nome del file non è nell'inode ma nella directory.

3.2 La system call stat()

`stat()` permette di ottenere informazioni dettagliate su un file riempiendo una struttura `struct stat`:

```
1 #include <sys/stat.h>
2
3 struct stat buf;
4 int err = stat(argv[1], &buf);
5 if (err == -1) { perror("stat"); exit(255); }
6
7 printf("nlink %d\n", buf.st_nlink);    // numero hard link
8 printf("uid   %d\n", buf.st_uid);      // UID proprietario
9 printf("gid   %d\n", buf.st_gid);      // GID gruppo
10 printf("perm  %o\n", buf.st_mode & 0777); // permessi in ottale
11
12 // Tipo del file
13 switch (buf.st_mode & S_IFMT) {
14     case S_IFBLK:  printf("block device\n"); break;
15     case S_IFCHR:  printf("character device\n"); break;
16     case S_IFDIR:  printf("directory\n"); break;
17     case S_IFIFO:  printf("FIFO/pipe\n"); break;
18     case S_IFLNK:  printf("symlink\n"); break;
19     case S_IFREG:  printf("regular file\n"); break;
20     case S_IFSOCK: printf("socket\n"); break;
21 }
```

Esempio pratico

```
$ ./prova_stat /etc/passwd
nlink 1
uid 0
gid 0
perm 644
regular file
```


3.3 stat() vs lstat(): la differenza con i symlink

Syscall	Comportamento con un symlink
stat()	Segue il symlink → restituisce info sul file di destinazione
lstat()	NON segue il symlink → restituisce info sul link stesso

```
$ ln -s /etc/passwd link_passwd # creo un symlink

$ ./prova_stat link_passwd      # usa stat() -> segue il link
perm 644                       # info di /etc/passwd
regular file

$ ./prova_lstat link_passwd     # usa lstat() -> info del link
perm 777                       # info del link simbolico stesso
symlink
```

3.4 Lettura delle directory con dirent

Le directory in UNIX sono file speciali che contengono una lista di voci. La libreria <dirent.h> fornisce le funzioni per leggerle.

3.4.1 readdir(): versione minimale di ls

```
1 #include <dirent.h>
2
3 int main() {
4     DIR *d = opendir(".");      // apre la directory corrente
5     struct dirent *de;
6     while ((de = readdir(d)) != NULL)
7         printf("%s\n", de->d_name); // stampa ogni nome
8     closedir(d);
9 }
```

- `opendir(path)`: apre la directory e restituisce un puntatore `DIR*`
- `readdir(dirp)`: restituisce la prossima voce come `struct dirent*`; ritorna `NULL` a fine directory
- `closedir(dirp)`: chiude e libera le risorse

La struttura `struct dirent` contiene almeno:

```
1 struct dirent {
2     ino_t  d_ino;      // numero di inode
3     char   d_name[];   // nome del file (max 255 caratteri su Linux)
4     // su Linux/glibc anche:
5     unsigned char d_type; // DT_REG, DT_DIR, DT_LNK, ...
6 };
```

Nota

L'ordine dei file restituiti da `readdir()` **non è alfabetico**: dipende da come il filesystem li memorizza internamente. Per avere un ordine, usare `scandir()`.

3.4.2 scandir(): lettura ordinata

```

1 #define _DEFAULT_SOURCE
2 #include <dirent.h>
3
4 int main(void) {
5     struct dirent **namelist;
6     int n = scandir(".", &namelist, NULL, alphasort);
7     if (n == -1) { perror("scandir"); exit(EXIT_FAILURE); }
8
9     for (int i = 0; i < n; i++) {
10         printf("%s\n", namelist[i]->d_name);
11         free(namelist[i]); // bisogna liberare ogni elemento
12     }
13     free(namelist);        // e l'array
14 }

```

Esempio pratico

```

$ ./scandir_test
.
..
documenti
file.txt      # ordinato alfabeticamente!
script.sh

```

Confronto tra readdir e scandir:

	readdir	scandir
Memoria	O(1), una voce per volta	O(N), tutto in memoria
Ordinamento	No	Sì (con <code>alphasort</code>)
Filtri	No	Sì (funzione filtro personalizzata)
Ideale per	Directory grandi	Directory piccole, output ordinato

4 Le System Call in UNIX: Gestione dei File

4.1 Il concetto di File Descriptor

Concetto chiave

In UNIX, ogni file aperto da un processo è rappresentato da un **file descriptor** (FD): un intero non negativo usato dal kernel per identificare in modo univoco il file all'interno del processo. È un "handle": il processo lo usa senza dover conoscere la struttura interna del filesystem.

Ogni processo inizia con 3 FD già aperti:

FD	Nome	Descrizione
0	stdin	Input standard (tastiera)
1	stdout	Output standard (terminale)
2	stderr	Errori standard (terminale)

Ogni nuovo file aperto con `open()` riceve il **più piccolo FD disponibile** ≥ 3 . Il concetto di FD si applica non solo ai file ma anche a pipe, socket, terminali e altri dispositivi.

4.2 Le syscall fondamentali: open, read, write, close

```

1 // Apre un file; restituisce il FD o -1 in caso di errore
2 int fd = open(pathname, flags);           // 2 parametri: apre
    esistente
3 int fd = open(pathname, flags, mode);      // 3 parametri: crea
    nuovo file
4
5 // Legge dati dal file in buf; restituisce bytes letti, 0=EOF, -1=
    errore
6 ssize_t n = read(fd, buf, count);
7
8 // Scrive count bytes da buf sul file; restituisce bytes scritti
9 ssize_t n = write(fd, buf, count);
10
11 // Chiude il FD liberando le risorse; restituisce 0 o -1
12 int status = close(fd);

```

4.2.1 Esempio completo: copia di un file

```

1 #include <fcntl.h>
2 #include <unistd.h>
3 #define BUFSIZE 1024
4
5 int main(int argc, char *argv[]) {
6     // Apre il file sorgente in sola lettura
7     int fdin = open(argv[1], O_RDONLY);
8     // Apre/crea il file destinazione in scrittura
9     int fdout = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0666);

```

1. Perché passiamo un numero a read e write?

Immagina di andare in banca. Prendi un bigliettino numerato (il File Descriptor). Tu non sai come funziona la cassaforte, né come il cassiere conta i soldi; a te basta andare allo sportello e dire: "Voglio depositare 100 euro sul conto associato al biglietto numero 3".

È esattamente quello che fanno i programmi in C:

- Il File Descriptor è solo un "handle", un'etichetta.
- Le funzioni `read()` e `write()` sono **system call**, ovvero le uniche "porte" per parlare con il cassiere (il Kernel).
- Il tuo processo usa il numero senza doversi minimamente preoccupare di come il filesystem gestisce i dischi, i settori o l'hardware.
- Quando scrivi `write(3, "ciao", 4)`, stai dicendo al kernel: "Ehi kernel, prendi questi 4 byte ('ciao') e vedi tu come scriverli sul dispositivo associato all'identificativo 3". Il kernel legge il numero 3, controlla la sua tabella segreta, vede che corrisponde (ad esempio) a una penna USB, e fa il lavoro sporco.

```

10
11 unsigned char buf[BUFSIZE];
12 ssize_t n;
13
14 // Legge blocchi di BUFSIZE byte e li scrive
15 while ((n = read(fdin, buf, BUFSIZE)) > 0)
16     write(fdout, buf, n);
17
18 close(fdin);
19 close(fdout);
20 }

```

Listing 5: Copia di file con open/read/write/close

Nota

Perché usare un buffer? Ogni chiamata `read()` o `write()` è una syscall costosa (richiede un context switch utente↔kernel). Usare un buffer da 1KB invece di leggere byte per byte riduce il numero di syscall di un fattore 1024, rendendo la copia enormemente più veloce.

4.3 `lseek()`: spostarsi nel file e i file sparsi

`lseek()` sposta la **posizione corrente** (offset) all'interno del file:

```

1 off_t lseek(int fd, off_t offset, int whence);
2 // whence: SEEK_SET (dall'inizio), SEEK_CUR (posizione corrente),
3 //          SEEK_END (dalla fine del file)
4
5 // Esempio: crea un file con un "buco"
6 int fdout = open(argv[1], O_WRONLY | O_TRUNC | O_CREAT, 0776);
7 write(fdout, "ciao\n", 5);           // scrive a offset 0, poi
8                                     // offset = 5
9 lseek(fdout, 102400, SEEK_SET);      // salta a offset 102400
10 write(fdout, "MARE\n", 5);          // scrive a offset 102400
11 close(fdout);

```

Offset	Contenuto	Note
0 – 4	"ciao"	Primo write
5 – 102399	0x00	"Buco" non allocato fisicamente
102400 – 102404	"MARE"	Secondo write

Il filesystem gestisce i buchi in modo efficiente: le zone vuote non vengono allocate su disco finché non vengono scritte. Questo è un **sparse file**.

4.4 `dup`, `dup2`, `dup3`: duplicazione e ridirezione

```
1 int newfd = dup(oldfd);           // newfd = piu' piccolo FD libero
2 int newfd = dup2(oldfd, newfd);  // chiude newfd se aperto, poi
    duplica
3 int newfd = dup3(oldfd, newfd, flags); // come dup2 + O_CLOEXEC
```

I FD duplicati condividono lo stesso oggetto file nel kernel: stesso offset, stessi flag di stato.

Ridirezione dell'output su un file:

```
1 int main(int argc, char *argv[]) {
2     // Apre il file di output
3     int fd = open(argv[1], O_WRONLY | O_TRUNC | O_CREAT, 0666);
4     // FD 1 (stdout) ora punta al file
5     dup2(fd, STDOUT_FILENO);
6     close(fd); // chiude il fd originale; la ridirezione resta
    attiva
7     printf("HELLO WORLD %d\n", 42); // finisce nel file, non nel
    terminale!
8 }
```

Listing 6: Ridirezione di stdout su file con dup2

```
$ ./hw out
$ cat out
HELLO WORLD 42
```

Questo è esattamente il meccanismo che le shell usano per implementare la ridirezione `>` e `<`.

4.5 File aperti dopo fork() e exec()

Dopo fork(): il figlio eredita una copia della file descriptor table del padre. I FD duplicati puntano allo stesso oggetto file nel kernel, condividendo l'offset corrente. Padre e figlio possono interlacciare le scritture se non sincronizzati.

Dopo exec(): di default tutti i FD sopravvivono e sono accessibili nel nuovo programma. Per evitarlo si imposta `FD_CLOEXEC`:

```
1 int fd = open("file.txt", O_RDONLY | O_CLOEXEC);
2 // oppure:
3 fcntl(fd, F_SETFD, FD_CLOEXEC); // il fd verra' chiuso alla exec
```

4.6 Le pipe

Concetto chiave

Una **pipe** è un canale di comunicazione unidirezionale tra processi. I dati scritti nell'estremità di scrittura (`fd[1]`) possono essere letti dall'estremità di lettura (`fd[0]`). Implementata dal kernel come un buffer circolare (tipicamente 64KB).

4.6.1 Pipe semplice

```
1 #include <unistd.h>
2
3 int fd[2];
4 pipe(fd);    // fd[0] = lettura, fd[1] = scrittura
5
6 write(fd[1], "ciao\n", 5);    // scrive nella pipe
7 ssize_t n = read(fd[0], buf, 1024); // legge dalla pipe
8 write(STDOUT_FILENO, buf, n); // stampa a schermo
```

4.6.2 Pipe con fork e ridirezione: implementare `ls | sort -r`

```
1 #include <unistd.h>
2 #include <sys/wait.h>
3
4 int main() {
5     int fd[2];
6     pipe(fd);    // crea la pipe: fd[0]=lettura, fd[1]=scrittura
7
8     // PRIMO FIGLIO: esegue "ls", reindirige stdout nella pipe
9     switch (fork()) {
10         case 0:
11             dup2(fd[1], STDOUT_FILENO); // ls scrive nella pipe
12             close(fd[0]);                // non legge dalla pipe
13             close(fd[1]);
14             execlp("ls", "ls", NULL);
15             exit(1);
16         default: break;
17         case -1: return 1;
18     }
19
20     // SECONDO FIGLIO: esegue "sort -r", legge stdin dalla pipe
21     switch (fork()) {
22         case 0:
23             dup2(fd[0], STDIN_FILENO); // sort legge dalla pipe
24             close(fd[0]);
25             close(fd[1]);                // non scrive nella pipe
26             execlp("sort", "sort", "-r", NULL);
27             exit(1);
28         default: break;
29         case -1: return 1;
30     }
31
32     // PADRE: chiude entrambi i fd (FONDAMENTALE!)
33     // Senza questa chiusura, sort non riceve mai EOF
34     close(fd[0]);
35     close(fd[1]);
36
37     int status;
```

```
38     wait(&status);
39     wait(&status);
40     return 0;
41 }
```

Listing 7: Implementazione manuale di `ls | sort -r`

Nota

È **fondamentale** che il padre chiuda entrambe le estremità della pipe. Se lasciasse aperto `fd[1]`, `sort` non riceverebbe mai EOF (ci sarebbe ancora almeno un “scrittore” aperto, cioè il padre) e resterebbe bloccato indefinitamente in attesa di altri dati.

4.7 ioctl: controllo dei dispositivi

```
1 int status = ioctl(fd, request, arg);
```

`ioctl()` è il meccanismo generico per inviare comandi specifici ai driver di dispositivo, andando oltre le operazioni standard di `read/write`. Il codice `request` è specifico di ogni dispositivo (espulsione CD-ROM, dimensioni terminale, configurazione porte seriali, ecc.).

5 Introduzione ai Segnali

5.1 Cosa sono i segnali

I **segnali** sono un meccanismo di comunicazione asincrona tra processi (o tra kernel e processo), usati per notificare il verificarsi di eventi. Sono identificati da interi (es. 9, 15) ma si usano macro simboliche (es. `SIGKILL`, `SIGTERM`).

Poiché la gestione è **asincrona**, un segnale può arrivare in qualsiasi momento, interrompendo il normale flusso di esecuzione. Alla ricezione di un segnale, un processo può:

1. **Ignorarlo**: `SIG_IGN` (`SIGKILL` e `SIGSTOP` non possono essere ignorati)
2. **Gestirlo**: eseguire una funzione handler personalizzata
3. **Comportamento di default**: nella maggior parte dei casi, terminare il processo

5.2 kill: inviare segnali

Nota

Il nome “kill” è fuorviante: non serve solo a terminare processi, ma a inviare **qualsiasi segnale** a qualsiasi processo. Si chiama così solo perché molti segnali, per default, terminano il processo.

```
1 #include <signal.h>
2
3 int kill(pid_t pid, int sig);
4 // Ritorna 0 in caso di successo, -1 in caso di errore
```

```

5 // errno = EPERM se non si hanno i permessi
6 // errno = ESRCH se il processo non esiste

1 int main(int argc, char *argv[]) {
2     if (argc < 3) { printf("Uso: %s <pid> <signal>\n", argv[0]);
3         return 1; }
4
5     pid_t pid = atoi(argv[1]);
6     int sig = atoi(argv[2]);
7
8     int rv = kill(pid, sig);
9     if (rv < 0) { perror("Errore nell'invio"); return 1; }
10    printf("Segnale %d inviato al PID %d\n", sig, pid);
11    return 0;
12 }

```

Listing 8: Reimplementazione del comando kill

5.3 signal: registrare un handler

Nota

Il nome “signal” è fuorviante: non invia un segnale, ma **registra una funzione di callback (handler)** da chiamare quando arriva un determinato segnale.

```

1 // Firma semplificata:
2 void (*signal(int signum, void (*handler)(int)))(int);
3 // handler puo' essere:
4 //   una funzione: void mio_handler(int signo) { ... }
5 //   SIG_IGN: ignora il segnale
6 //   SIG_DFL: ripristina il comportamento di default

1 void handler(int signo) {
2     printf("\n--> INTERRUPT! Ricevuto segnale %d\n", signo);
3     // Quando questa funzione termina, il controllo torna al main
4 }

5
6 int main() {
7     printf("Il mio PID %d. Mandami un SIGUSR1!\n", getpid());
8     signal(SIGUSR1, handler); // registra l'handler
9
10    for (int i = 0;; i++) {
11        printf("Main loop: waiting %d\n", i);
12        sleep(2); // NOTA: sleep viene interrotta immediatamente
13                  // se arriva un segnale!
14    }
15 }

```

Listing 9: Gestione asincrona base: intercettare SIGUSR1

Flusso di esecuzione quando arriva il segnale:

1. Il kernel ferma l'esecuzione del `main` (anche nel mezzo di una `sleep`)
2. Esegue la funzione `handler`
3. Quando `handler` termina, il controllo torna al `main`
4. La `sleep` interrotta **non riprende**: ritorna immediatamente

5.4 Segnali da eccezioni hardware

5.4.1 SIGSEGV — Segmentation Fault

Generato dalla MMU quando il processo accede a memoria non valida (puntatore NULL, fuori dai segmenti allocati, memoria di sola lettura).

```

1 void handler(int signo) {
2     printf("Errore critico: Segmentation Fault (%d)\n", signo);
3     printf("Terminazione controllata...\n");
4     exit(1); // FONDAMENTALE: senza exit, tornerebbe all'
              // istruzione
              // che ha causato il fault -> loop infinito!
5 }
6
7
8 int main() {
9     signal(SIGSEGV, handler);
10    int *p = NULL;
11    *p = 42; // causa SIGSEGV: la CPU tenta di scrivere a
              // indirizzo 0
12 }

```

Listing 10: Gestione controllata di SIGSEGV

Concetto chiave

Quando l'handler di SIGSEGV termina senza chiamare `exit()`, il sistema operativo ripristina l'esecuzione **riavviando l'istruzione che ha causato il fault**. Poiché il problema non è stato risolto, il fault si ripete, l'handler viene chiamato di nuovo, e così via: **loop infinito**.

5.4.2 SIGFPE — Errore aritmetico

Generato in caso di divisione per zero o overflow (nonostante il nome "Floating Point", viene generato anche per operazioni intere).

```

1 int den = 0; // variabile globale modificabile dall'handler
2
3 void handler(int signo) {
4     printf("Errore matematico: correggo den da 0 a 2\n");
5     den = 2; // corregge la causa dell'errore
6     // L'handler ritorna; il sistema riesegue l'istruzione fallita
7     // Questa volta: 42 / 2 = 21 -> successo!
8 }
9

```

```

10 int main() {
11     signal(SIGFPE, handler);
12     int risultato = 42 / den; // 1a esecuzione: fault -> handler
13                               // 2a esecuzione: 42/2 = 21 -> ok
14     printf("Risultato: %d\n", risultato); // stampa: 21
15     return 0;
16 }

```

Listing 11: Tentativo di recovery da divisione per zero

5.4.3 SIGINT — Intercettare Ctrl+C

```

1 void handler(int signo) {
2     printf("\nRicevuto segnale %d... Non mi chiudo!\n", signo);
3 }
4
5 int main() {
6     signal(SIGINT, handler); // override di Ctrl+C
7     for (int i = 0;; i++) {
8         printf("waiting %d\n", i);
9         sleep(1);
10    }
11    // Per terminare questo processo bisogna usare:
12    // Ctrl+\ (invia SIGQUIT, non intercettato)
13    // kill -9 <pid> (SIGKILL non puo' essere intercettato)
14 }

```

5.5 sigaction: gestione avanzata dei segnali

sigaction() è più flessibile e portabile di signal(). Permette in più:

- SA_SIGINFO: l'handler riceve una struttura siginfo_t con informazioni dettagliate sul mittente (PID, UID, causa del segnale)
- sa_mask: blocca altri segnali durante l'esecuzione dell'handler, evitando race condition
- SA_RESTART: riavvia automaticamente le syscall bloccanti interrotte dal segnale
- SA_RESETHAND: ripristina il comportamento di default dopo la prima esecuzione dell'handler

```

1 // Handler esteso: riceve anche i dettagli del mittente
2 void handler(int signo, siginfo_t *si, void *ptr) {
3     printf("Ricevuto segnale %d\n", signo);
4     printf("Mittente PID: %d. Invio SIGKILL al mittente!\n", si->
5         si_pid);
6     if (si->si_pid > 0)
7         kill(si->si_pid, 9); // uccide chi ha inviato il segnale!
8 }

```

```

9 int main() {
10     struct sigaction sa = {
11         .sa_sigaction = handler,    // handler a 3 argomenti
12         .sa_flags      = SA_SIGINFO // abilita siginfo_t
13     };
14
15     printf("Mio PID: %d. In attesa...\n", getpid());
16     sigaction(SIGUSR1, &sa, NULL);
17
18     for (;;) pause(); // attende segnali in modo efficiente (non
19                       // spreca CPU)
20 }

```

Listing 12: Il “contrattacco”: sigaction con SA_SIGINFO

La struttura `siginfo_t` contiene tra l'altro:

- `si->si_pid`: PID del mittente del segnale
- `si->si_uid`: UID del mittente
- `si->si_code`: causa specifica del segnale

6 Attesa di eventi: I/O Multiplexing

6.1 Il problema: blocco su read

Se un processo deve leggere da più sorgenti (`stdin`, un file, una socket), non può semplicemente fare `read()` su ciascuna: la prima sorgente senza dati **blocca il processo indefinitamente**, impedendo di controllare le altre.

La soluzione è l'**I/O multiplexing**: una syscall che sospende il processo finché **almeno uno** dei file descriptor monitorati ha dati pronti.

6.2 Evoluzione storica

Syscall	Epoca	Caratteristiche
<code>select/pselect</code>	Storica	Limite fisso a 1024 FD; usa bitmask inefficienti
<code>poll/ppoll</code>	Moderna	Array dinamico; nessun limite fisso al numero di FD
<code>epoll</code>	Linux	$O(1)$; scala a migliaia di connessioni

6.3 La system call `poll()`

`poll()` usa un array di strutture `pollfd`:

```

1 struct pollfd {
2     int    fd;        // file descriptor da monitorare
3     short  events;    // eventi richiesti (input: cosa vogliamo
                        // monitorare)
4     short  revents;   // eventi accaduti (output: cosa ha segnalato il
                        // kernel)
5 };

```

I principali eventi:

- POLLIN: ci sono dati da leggere
- POLLOUT: è possibile scrivere senza bloccarsi
- POLLHUP: l'altra estremità ha chiuso la connessione
- POLLERR: errore sul file descriptor

6.4 Esempio completo: bridge tra file e standard I/O

Questo programma implementa un “ponte”: legge da stdin e scrive su un file, e contemporaneamente legge da un altro file e scrive su stdout.

```

1 #include <poll.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4
5 char buf[1024];
6
7 int main(int argc, char *argv[]) {
8     // O_NONBLOCK: evita blocchi anche in caso di falsi positivi di
9     // poll
10    int fdin = open(argv[1], O_RDONLY | O_NONBLOCK);
11    int fdout = open(argv[2], O_WRONLY);
12
13    // Array di 2 FD da monitorare
14    struct pollfd pfd[] = {
15        {0, POLLIN, 0}, // indice 0: monitora stdin per lettura
16        {fdin, POLLIN, 0} // indice 1: monitora fdin per lettura
17    };
18
19    for (;;) {
20        // Sospende il processo finché uno dei due FD ha eventi
21        // timeout = -1: attesa infinita (non consuma CPU)
22        if (poll(pfd, 2, -1) < 0) { perror("poll"); return 1; }
23
24        // Controlla stdin (pfd[0]) QUALITIC SU TASTIERA
25        if (pfd[0].revents & POLLIN) {
26            int n = read(0, buf, sizeof(buf));
27            if (n == 0) return 0; // EOF su stdin (Ctrl+D)
28            write(fdout, buf, n); // scrive nel file di output
29        }
30
31        // Controlla fdin (pfd[1]) DATI NBL FILE
32        if (pfd[1].revents & POLLHUP) return 0; // altra estremità
33        // chiusa
34        if (pfd[1].revents & POLLIN) {
35            int n = read(fdin, buf, sizeof(buf));
36            if (n == 0) return 0; // EOF
37            write(1, buf, n); // scrive su stdout
38        }
39    }
40 }
```

```
37     }  
38 }
```

Listing 13: Bridge bidirezionale con poll()

Analisi del flusso:

1. `poll(pfd, 2, -1)`: il processo si addormenta qui, senza consumare CPU
2. Quando arriva un dato su `stdin` o su `fdin`, il kernel risveglia il processo
3. Il programma controlla `.revents` (scritto dal kernel) con AND bit a bit
4. Esegue la lettura e scrittura appropriata
5. Torna ad aspettare

Nota

Perché `O_NONBLOCK` su `fdin`? `poll()` potrebbe in rari casi segnalare un FD come pronto per la lettura ma la successiva `read()` potrebbe comunque bloccarsi (race condition). Con `O_NONBLOCK`, la `read()` non si blocca mai: ritorna immediatamente con `EAGAIN` se non ci sono dati.